Computer Organization and Architecture Lab

Verilog Implementation of Adders, Multipliers, and ALU Blocks for Processor Design

Faculty Development Program

IIT Indore

Lab Duration:	3 Hours
Material:	Verilog HDL source files and one self-contained lab handout
Prepared by:	Mrunal Shende
Department:	Computer Science and Engineering, IIT Indore

Lab Theme: From basic arithmetic circuits to an integrated ALU datapath block

Contents

1. Aim of the Lab

Objective

The aim of this lab is to implement basic arithmetic and logic blocks used in processor datapaths using Verilog HDL. The lab begins with half adder and full adder circuits, builds a 4-bit ripple carry adder, implements a 4-bit multiplier, and finally integrates arithmetic and logical operations into a 4-bit ALU.

Learning Outcomes

After completing this lab, participants will be able to:

- understand the role of adders, multipliers, and ALUs in processor datapaths;
- write Verilog modules for combinational circuits;
- build larger arithmetic blocks using smaller modules;
- understand how simple testbench inputs are applied over time;
- verify expected outputs using truth tables and Boolean equations.

2. Three-Hour Lab Plan

Time	Activity
0:00–0:15	Introduction to processor datapath blocks and Verilog basics
0:15–0:45	Experiment 1: Half adder and full adder implementation
0:45–1:20	Experiment 2: 4-bit ripple carry adder implementation
1:20–1:55	Experiment 3: 4-bit multiplier implementation
1:55–2:35	Experiment 4: Gate-level 4-bit ALU implementation
2:35–2:50	Review of Verilog files, testbench inputs, and expected results
2:50–3:00	Discussion, viva questions, and mini-assignment

Note

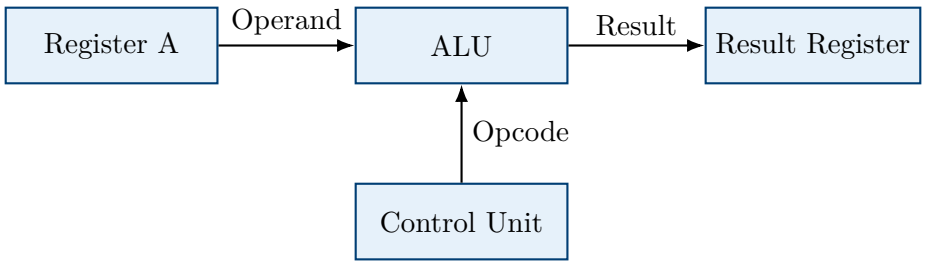
This lab is designed as a single self-contained handout. The same PDF can be projected during the session and shared with participants for hands-on implementation.

Suggested Presentation Flow

1. Explain why adders, multipliers, and ALUs are basic datapath blocks.
2. Start from Boolean expressions and then show the corresponding Verilog `assign` statements.
3. Show the associated testbench inputs and ask participants to predict the outputs.
4. For the ALU, first explain opcode decoding, then explain output selection using simple AND-OR logic.

3. Background: Processor Datapath Blocks

A processor datapath contains the hardware blocks that perform arithmetic, logic, data movement, and storage operations. In a basic processor, the ALU is one of the central datapath components. It receives operands from registers, performs an operation selected by a control signal, and produces a result.



Main Components Covered in This Lab

- **Adder:** Performs binary addition.
- **Multiplier:** Performs binary multiplication.
- **ALU:** Selects and performs arithmetic or logical operations based on an opcode.

4. Verilog Basics Required for This Lab

A Verilog design is written as a `module`. A module has input ports, output ports, internal wires, and logic statements. In this lab, design modules are written using `wire` and `assign`, so the hardware structure remains close to the Boolean equations.

```
module example_gate(
    input  wire a,
    input  wire b,
    output wire y
);

assign y = a & b;

endmodule
```

Listing 1: Basic Verilog module template

Important Verilog Constructs

- `assign`: used for continuous assignment in combinational logic.
- `wire`: used for connections between logic blocks.
- `reg`: used only in the testbench here, because testbench input values change with time.
- `#10`: used in the testbench to wait for 10 ns before applying the next input.

5. Experiment 1: Half Adder and Full Adder

5.1 Half Adder

A half adder adds two one-bit inputs A and B. It produces a sum bit and a carry bit.

$$Sum = A \oplus B = A'B + AB', \qquad Carry = A \cdot B$$

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

```
`timescale 1ns/1ps

module half_adder(
    input  wire a,
    input  wire b,
    output wire sum,
    output wire carry
);

assign sum   = (~a & b) | (a & ~b);
assign carry = a & b;

endmodule
```

Listing 2: Verilog code for half adder

5.2 Full Adder

A full adder adds three one-bit inputs: A, B, and C_{in}.

$$Sum = A \oplus B \oplus C_{in}$$

$$Sum = A'B'C_{in} + A'BC'_{in} + AB'C'_{in} + ABC_{in}$$

$$Carry = AB + AC_{in} + BC_{in}$$

```
`timescale 1ns/1ps

module full_adder(
    input  wire a,
    input  wire b,
    input  wire cin,
    output wire sum,
    output wire cout
);

assign sum  = (~a & ~b & cin) |
              (~a &  b & ~cin) |
              ( a & ~b & ~cin) |
              ( a &  b & cin);
assign cout = (a & b) | (b & cin) | (a & cin);

endmodule
```

Listing 3: Verilog code for full adder

5.3 Testbench for Full Adder

```
`timescale 1ns/1ps

module tb_full_adder;

reg a;
reg b;
reg cin;
wire sum;
wire cout;
```

```
10
11 full_adder uut(
12     .a(a),
13     .b(b),
14     .cin(cin),
15     .sum(sum),
16     .cout(cout)
17 );
18
19 initial begin
20     a = 0; b = 0; cin = 0; #10;
21     a = 0; b = 0; cin = 1; #10;
22     a = 0; b = 1; cin = 0; #10;
23     a = 0; b = 1; cin = 1; #10;
24     a = 1; b = 0; cin = 0; #10;
25     a = 1; b = 0; cin = 1; #10;
26     a = 1; b = 1; cin = 0; #10;
27     a = 1; b = 1; cin = 1; #10;
28
29     $finish;
30 end
31
32 endmodule
```

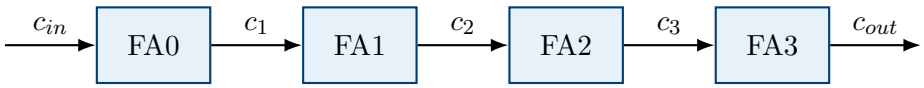
Listing 4: Testbench for full adder

Lab Exercise

For each input combination, use the full adder truth table to verify `sum` and `cout`.

6. Experiment 2: 4-bit Ripple Carry Adder

A ripple carry adder is built by connecting multiple full adders in series. The carry output of one full adder becomes the carry input of the next full adder.




```
1  `timescale 1ns/1ps
2
3  module ripple_carry_adder_4bit(
4      input  wire [3:0] a,
5      input  wire [3:0] b,
6      input  wire       cin,
7      output wire [3:0] sum,
8      output wire       cout
9  );
10
11 wire c1, c2, c3;
12
13 full_adder fa0(
14     .a(a[0]),
15     .b(b[0]),
16     .cin(cin),
17     .sum(sum[0]),
18     .cout(c1)
19 );
20
21 full_adder fa1(
22     .a(a[1]),
23     .b(b[1]),
24     .cin(c1),
25     .sum(sum[1]),
26     .cout(c2)
27 );
28
29 full_adder fa2(
30     .a(a[2]),
31     .b(b[2]),
32     .cin(c2),
```

```
    .sum(sum[2]),
    .cout(c3)
);

full_adder fa3(
    .a(a[3]),
    .b(b[3]),
    .cin(c3),
    .sum(sum[3]),
    .cout(cout)
);

endmodule
```

Listing 5: 4-bit ripple carry adder using full adder modules

6.1 Testbench for 4-bit Ripple Carry Adder

```
`timescale 1ns/1ps

module tb_ripple_carry_adder_4bit;

reg  [3:0] a;
reg  [3:0] b;
reg        cin;
wire [3:0] sum;
wire       cout;

ripple_carry_adder_4bit uut(
    .a(a),
    .b(b),
    .cin(cin),
    .sum(sum),
    .cout(cout)
);

initial begin
    a = 4'd3;  b = 4'd5; cin = 0; #10;
    a = 4'd7;  b = 4'd8; cin = 0; #10;
    a = 4'd15; b = 4'd1; cin = 0; #10;
    a = 4'd10; b = 4'd6; cin = 1; #10;

    $finish;
end

endmodule
```

Listing 6: Testbench for 4-bit ripple carry adder

Discussion Point

The delay of a ripple carry adder depends on carry propagation. The carry must travel from the least significant bit to the most significant bit. This makes the circuit simple, but not the fastest option for large bit-widths.

7. Experiment 3: 4-bit Multiplier

A multiplier produces the product of two binary numbers. For two 4-bit inputs, the maximum value is:

$$15 \times 15 = 225$$

Therefore, the product requires 8 bits.

```
`timescale 1ns/1ps

module multiplier_4bit(
    input  wire [3:0] a,
    input  wire [3:0] b,
    output wire [7:0] product
```

```
);

assign product = a * b;

endmodule
```

Listing 7: 4-bit combinational multiplier

7.1 Testbench for 4-bit Multiplier

```
`timescale 1ns/1ps

module tb_multiplier_4bit;

reg  [3:0] a;
reg  [3:0] b;
wire [7:0] product;

multiplier_4bit uut(
    .a(a),
    .b(b),
    .product(product)
);

initial begin
    a = 4'd3;  b = 4'd4;  #10;
    a = 4'd7;  b = 4'd2;  #10;
    a = 4'd15; b = 4'd15; #10;
    a = 4'd9;  b = 4'd6;  #10;

    $finish;
end

endmodule
```

Listing 8: Testbench for 4-bit multiplier

Note

In this lab, the multiplication operator * is used to keep the implementation simple within the 3-hour session. Internally, hardware multiplication can be implemented using partial products, shift operations, and adders.

Lab Exercise

As an extension, implement a 4-bit array multiplier using AND gates for partial products and adders for summation.

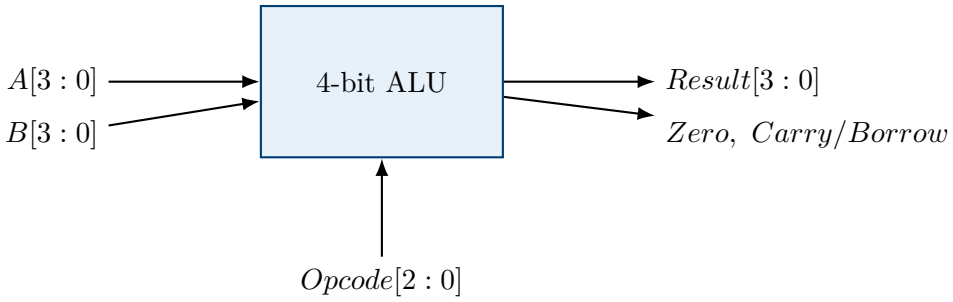
8. Experiment 4: 4-bit ALU Design

An Arithmetic Logic Unit performs arithmetic and logical operations based on a control input called an opcode.

8.1 ALU Operation Table

Opcode	Operation	Description
000	ADD	$A + B$
001	SUB	$A - B$
010	AND	Bitwise AND
011	OR	Bitwise OR
100	Student task 1	Unimplemented in starter code
101	Student task 2	Unimplemented in starter code
110	Student task 3	Unimplemented in starter code
111	Student task 4	Unimplemented in starter code

8.2 ALU Block Diagram



8.3 Verilog Code for 4-bit ALU

Note

This starter ALU is written using structural/dataflow Verilog. It does not use `always`, `case`, or output `reg`. The starter code implements only ADD, SUB, AND, and OR. Opcodes 100–111 are intentionally left unimplemented for the assignment task.

```
`timescale 1ns/1ps

module alu_full_adder(
    input wire a, b, cin,
    output wire sum, cout
);

assign sum  = (~a & ~b & cin) |
              (~a &  b & ~cin) |
              ( a & ~b & ~cin) |
              ( a &  b & cin);
assign cout = (a & b) | (b & cin) | (a & cin);

endmodule

module alu_4bit(
    input  wire [3:0] a,
    input  wire [3:0] b,
    input  wire [2:0] opcode,
    output wire [3:0] result,
    output wire       carry,
    output wire       zero
);

wire op2_not, op1_not, op0_not;
wire sel_add, sel_sub, sel_and, sel_or;

wire [3:0] add_result, sub_result;
wire [3:0] and_result, or_result;
```

```

wire add_c1, add_c2, add_c3, add_cout;
wire sub_c1, sub_c2, sub_c3, sub_cout, sub_borrow;

assign op2_not = ~opcode[2];
assign op1_not = ~opcode[1];
assign op0_not = ~opcode[0];

assign sel_add = op2_not & op1_not & op0_not;
assign sel_sub = op2_not & op1_not & opcode[0];
assign sel_and = op2_not & opcode[1] & op0_not;
assign sel_or  = op2_not & opcode[1] & opcode[0];

alu_full_adder add0(a[0], b[0], 1'b0, add_result[0], add_c1);
alu_full_adder add1(a[1], b[1], add_c1, add_result[1], add_c2);
alu_full_adder add2(a[2], b[2], add_c2, add_result[2], add_c3);
alu_full_adder add3(a[3], b[3], add_c3, add_result[3], add_cout);

alu_full_adder sub0(a[0], ~b[0], 1'b1, sub_result[0], sub_c1);
alu_full_adder sub1(a[1], ~b[1], sub_c1, sub_result[1], sub_c2);
alu_full_adder sub2(a[2], ~b[2], sub_c2, sub_result[2], sub_c3);
alu_full_adder sub3(a[3], ~b[3], sub_c3, sub_result[3], sub_cout);

assign sub_borrow = ~sub_cout;

assign and_result = a & b;
assign or_result  = a | b;

assign result = ({4{sel_add}} & add_result) |
                ({4{sel_sub}} & sub_result) |
                ({4{sel_and}} & and_result) |
                ({4{sel_or}}  & or_result);

assign carry = (sel_add & add_cout) |
               (sel_sub & sub_borrow);

assign zero = ~(result[0] | result[1] | result[2] | result[3]);

endmodule
```

Listing 9: Gate-level 4-bit ALU design

8.4 Testbench for 4-bit ALU

```
`timescale 1ns/1ps

module tb_alu_4bit;

reg  [3:0] a;
reg  [3:0] b;
reg  [2:0] opcode;
wire [3:0] result;
wire       carry;
wire       zero;

alu_4bit uut(
    .a(a),
    .b(b),
    .opcode(opcode),
    .result(result),
    .carry(carry),
    .zero(zero)
);

initial begin
    a = 4'd7; b = 4'd3;

    opcode = 3'b000; #10;
    opcode = 3'b001; #10;
    opcode = 3'b010; #10;
    opcode = 3'b011; #10;

    a = 4'd15; b = 4'd1;
    opcode = 3'b000; #10;

```

```
    $finish;
end

endmodule
```

Listing 10: Testbench for 4-bit ALU

Discussion Point

The ALU uses the opcode to select one operation from multiple arithmetic and logic operations. In an actual processor, the opcode is generated by the control unit after decoding the instruction.

9. Files Provided for the Lab

Each experiment has a Verilog design file and a small associated testbench file. The testbench only applies input values over time. It does not use display tables or tool-specific commands.

- **Half adder and full adder:** `half_adder.v`, `full_adder.v`, `tb_half_adder.v`, `tb_full_adder.v`
- **Ripple carry adder:** `full_adder.v`, `ripple_carry_adder_4bit.v`

  Testbench: `tb_ripple_carry_adder_4bit.v`
- **Multiplier:** `multiplier_4bit.v`, `tb_multiplier_4bit.v`
- **ALU:** `alu_4bit.v`, `tb_alu_4bit.v`

10. Common Debugging Issues

Issue	Possible Reason
Output remains unknown `x`	Input values may not be initialized in the testbench
Compilation error	Module name or port name mismatch
Wrong multiplier output	Product width may be too small
ALU result incorrect	Opcode mapping may be wrong
Carry/borrow not visible	Carry output may not be connected or displayed
Simulation does not stop	Missing `$finish` statement

11. Mini-Assignment

Lab Exercise

Complete any two of the unimplemented ALU opcode slots 100–111. You may choose from the following operations, or propose another operation to the instructor:

1. NAND
2. NOR
3. XOR
4. NOT of input A
5. Logical shift left or logical shift right of input A

12. Viva and Discussion Questions

1. Why is a full adder required in multi-bit addition?
2. What is the main limitation of a ripple carry adder?
3. Why does a 4-bit multiplier produce an 8-bit output?
4. What is the role of opcode in an ALU?
5. What is the difference between combinational and sequential logic?
6. Why do we need a testbench in HDL design?
7. How is the zero flag useful in processor design?
8. How can the ALU be extended for a real processor datapath?

13. Conclusion

In this lab, participants implemented basic arithmetic and logic blocks used in processor datapaths. The lab started with half adder and full adder circuits, extended them to a 4-bit ripple carry adder, introduced multiplier design, and finally integrated multiple operations into a 4-bit ALU. These blocks form the foundation of arithmetic and logic execution inside a processor.

End of Lab